	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name: N.Kruthika Reg. No:2503a52584
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 22/09/2026

1. Question:

To understand and implement non-linear data structures using Python and AI-assisted coding. Students will apply Tree, Graph, Heap, and Trie structures to practical problems and justify their choices using Big-O complexity.

- **TASK-1**

2. Prompt:

Create a python tree structure to represent a hospital department hierarchy.implement adding child nodes,displaying the hierarchy,searching for a department,and counting nodes.Explain how a tree is different from a list and provide test cases.

3. Code:

tree nodes.py > ...

```python
"""A tree representation of a hospital department hierarchy.

Unlike a list, which stores items in a single linear sequence, a tree stores
parent/child relationships.  This makes a hierarchy easy to display and
search by branches; a list does not directly represent those relationships.
"""


class DepartmentNode:
    """A department in the hospital hierarchy."""

    def __init__(self, name):
        self.name = name
        self.children = []

    def add_child(self, child):
        """Add and return a child department node."""
        if isinstance(child, str):
            child = DepartmentNode(child)
        if not isinstance(child, DepartmentNode):
            raise TypeError("child must be a DepartmentNode or department name")
        self.children.append(child)
        return child
```

4. **Output:**

City Hospital
Clinical Services
  Emergency
  Cardiology
Support Services
  Radiology
  Pharmacy
Node count: 7
All tests passed.

5. **Observation:**

The tree represents the hospital hierarchy naturally because each department can have child departments. Searching recursively checks the hierarchy and node counting visits each node. Unlike a simple list, a tree preserves parent-child relationships.
Challenge – 10,000 departments: A simple unbalanced tree can require O(n) search in the worst case. A balanced search tree can reduce search to about O(log n), depending on the tree structure.

- **TASK-2**

1. **Prompt:**

geneate a python graph structure to reresent hospital route finder .implement by adding hospital locations as nodes,connection as edges,displaying the graph by adjacent list, checking whether two locations are connected or not.implement the code by finding a route using BFS or DFS.explain why

graph more suitable than a simple list for representing connections between hospital locations and test at least three source-destination pairs

2. **Code:**

graph nodes.py > HospitalGraph

```python
"""A graph-based hospital route finder.

A graph is more suitable than a simple list because each location can connect
to multiple other locations, including routes that form cycles.  An adjacency
list stores these relationships directly, while a list would require manual
searching and would not naturally represent branching connections.
"""


from collections import deque


class HospitalGraph:
    """Undirected graph of hospital locations and walkable connections."""

    def __init__(self):
        self.adjacency = {}

    def add_location(self, location):
        """Add a location node if it is not already present."""
        self.adjacency.setdefault(location, set())

    def add_connection(self, first, second):
        """Add a bidirectional edge between two hospital locations."""
```

3. **Output:**

Cafeteria: Operating Rooms, Pharmacy
Emergency: Operating Rooms, Reception
Main Entrance: Reception
Operating Rooms: Cafeteria, Emergency
Pharmacy: Cafeteria, Radiology
Radiology: Pharmacy, Reception
Reception: Emergency, Main Entrance, Radiology
Main Entrance -> Operating Rooms: ['Main Entrance', 'Reception', 'Emergency', 'Operating Rooms']
Radiology -> Cafeteria: ['Radiology', 'Pharmacy', 'Cafeteria']
Emergency -> Unknown Ward: None
All route tests passed.

4. **Observation:**

A graph is more suitable than a list because hospital locations can have many-to-many connections. An adjacency list directly represents which locations are connected, and BFS can find a route through the network.

- **TASK-3**

1 Prompt:

create a python heap structure of Emergency Patient Priority.impement by addng a patient with a priority value,able to view the highest-priority patient, Remove/serve the highest-priority patient, displaying waiting patients, and able to handle at least one tie in priority appropriately. provide test cases of insertion and removal operations

2.Code:

heap nodes.py > ...

```python
"""Emergency patient priority queue implemented with a max-heap."""


class EmergencyPatient:
    def __init__(self, name, priority, arrival_order):
        self.name = name
        self.priority = priority
        self.arrival_order = arrival_order

    def __repr__(self):
        return f"{self.name} (priority={self.priority})"


class EmergencyPatientHeap:
    """Higher priority is served first; equal priorities are FIFO."""

    def __init__(self):
        self._heap = []
        self._next_arrival = 0

    @staticmethod
    def _higher_priority(first, second):
        return (first.priority, -first.arrival_order) > (
            second.priority,
            -second.arrival_order
```

3.Output:

```
Next patient: Bob (priority=5)
Waiting patients:
1. Bob (priority=5)
2. Cara (priority=5)
3. Alice (priority=2)
4. Dan (priority=1)
All tests passed.
```

4.Observation:

The heap keeps the smallest priority value at the top, so priority 1 patients are served first. The insertion order is stored as a secondary value, so patients with the same priority are handled consistently

- **TASK-4**

1 Prompt:

generate a python trie structure of Medical Word / Drug Search.Implement the code by createing a Trie, inserting medical words, searching for an exact word, checking whether a prefix exists or not, displaying/autocomplete words beginning with a prefix. Compare searching medical words using a list with searching using a Trie. Explain the advantages and disadvantages and provide test cases at least three different prefixes.

2.Code:

trie nodes.py > ...

```python
"""Trie-based medical word and drug search.

This example uses a small sample vocabulary for demonstration only; it is not
medical advice or a complete drug database.
"""

from dataclasses import dataclass, field
from timeit import timeit


@dataclass
class TrieNode:
    children: dict[str, "TrieNode"] = field(default_factory=dict)
    is_word: bool = False


class MedicalTrie:
    def __init__(self, words=()):
        self.root = TrieNode()
        for word in words:
            self.insert(word)

    @staticmethod
    def _normalise(text: str) -> str:
```

3.Output:

```
Exact searches:
  metformin: True
  Metformin: True
  vitamin: False

Prefix tests and autocomplete:
  'am': exists=True, matches=['amlodipine', 'amoxicillin']
  'met': exists=True, matches=['metformin', 'metoprolol']
  'car': exists=False, matches=[]
  'p': exists=True, matches=['paracetamol', 'penicillin', 'prednisone']

10,000 exact searches for 'metformin':
  list: 0.095208s; trie: 0.024862s
```

4.Observation:

List vs Trie: A list can be simple to implement, but prefix searching normally requires checking many words. A Trie stores characters by shared prefixes, making prefix lookup and autocomplete efficient. A Trie can use more memory because it creates many nodes and links. A list is simpler and can be preferable for small datasets.

TASK-5 – AI Decision Challenge: Choose the Correct Structure

The following choices match the practical purpose of each scenario.

Scenario	Your Choice + Reason	Main Operation	Typical Big-O
Hospital department hierarchy	Tree – represents parent-child hierarchy.	Search / traversal	O(n) general tree search
Finding a route between hospital rooms	Graph – represents connected locations and routes.	BFS/DFS	O(V + E)
Emergency patients based on severity	Heap – keeps highest-priority patient accessible.	Insert / remove priority	O(log n)
Autocomplete medical terms	Trie – stores shared prefixes efficiently	Prefix search	O(m) for prefix length m
Frequently finding the highest-priority task	t-priority taskHeap – highest-priority element is at the root.	Peek	O(1)

AI Recommendation and Comparison:

AI recommendation: Tree for hierarchy, Graph for routes, Heap for priority-based emergency patients, and Trie for medical-term autocomplete. These recommendations match the choices above because each structure directly supports the main relationship or operation in the scenario. No disagreement was identified.

Final Observation: Tree, Graph, Heap and Trie solve different types of non-linear data problems. The correct structure depends on whether the application mainly needs hierarchy, connections/routes, priority ordering, or prefix-based searching.

Conclusion: The programs were implemented and tested for the required non-linear data structures. The examples demonstrate their practical use in hospital software and show how Big-O complexity helps justify a data-structure choice.